

CHAPTER 1

INTRODUCTION

1.1. PURPOSE OF THIS STUDY

The purpose of this study is to design and develop Eventron, a modern, high-performance, full-stack event management and discovery platform. Traditional event management solutions often require organizers to use multiple fragmented tools for ticketing, marketing, attendee engagement, and analytics. This project aims to consolidate these processes into a unified platform by leveraging cutting-edge technologies, including Artificial Intelligence (AI) for content generation and vector-based recommendations, robust end-to-end encryption, and real-time bidirectional communications.

1.2. BRIEF OVERVIEW OF THE PROJECT REPORT

This report outlines the complete software development lifecycle of Eventron. It covers the literature review of existing solutions, the specific problems addressed by Eventron, and the project's core objectives. Furthermore, it details the feasibility study, system analysis, software engineering paradigms applied, system and database design, and concludes with the results, discussions, and potential future enhancements.

CHAPTER 2

LITERATURE OVERVIEW

A review of existing event management platforms (such as Eventbrite, Meetup, and Cvent) reveals several gaps in the market. While these platforms handle basic ticketing and registration, they often lack built-in AI capabilities to assist organizers with marketing content or to provide highly personalized attendee recommendations. Furthermore, advanced features like team-based registrations, digital holographic tickets, and real-time omni-channel notifications (Web Push, SMS, Email) are typically premium add-ons or non-existent. Eventron was conceptualized to address these limitations by integrating modern AI models (Google Gemini) and real-time web technologies directly into the core platform.

CHAPTER 3

PROBLEM DEFINITION & OBJECTIVES

PROBLEM DEFINITION

Organizers face significant friction when planning events, writing marketing copy, managing team registrations, and tracking real-time analytics. Conversely, attendees struggle to discover events that align perfectly with their interests and face cumbersome check-in processes at venues.

OBJECTIVES

1. **Unified Event Management:** Build an end-to-end platform for event creation, dynamic registration workflows, and real-time ticketing.
2. **AI Integration:** Utilize Google Gemini for an AI Content Studio (generating titles, descriptions, and agendas) and vector-based personalized event recommendations.
3. **Secure & Flexible Authentication:** Implement multi-modal authentication (Email/Password, Google OAuth, SMS OTP) and robust Role-Based Access Control (RBAC).
4. **Seamless Attendee Experience:** Provide digital holographic tickets with QR codes for fast on-site scanning and check-in.
5. **Integrated Payments:** Facilitate secure checkout processes using Razorpay with server-side signature verification.

CHAPTER 4

FEASIBILITY STUDY

- **Technical Feasibility:** Highly feasible. The system utilizes a modern, well-supported tech stack (React 19, FastAPI, MongoDB) and leverages reliable third-party APIs (Firebase, Twilio, Razorpay, Google Gemini) which are capable of handling high loads and complex data processing.
- **Economic Feasibility:** The project is economically viable as it utilizes cloud-based infrastructure (Render, MongoDB Atlas) with scalable pricing models, allowing it to operate efficiently with low initial overhead costs.
- **Operational Feasibility:** Designed with user experience in mind, the system clearly delineates roles between Admins, Organizers, and Attendees, ensuring that the platform is intuitive and operationally smooth for all stakeholders.

CHAPTER 5

SYSTEM ANALYSIS / PROPOSED SCHEME

The proposed scheme for Eventron follows a Single Page Application (SPA) architecture combined with a centralized, asynchronous RESTful API and WebSocket server.

- **Frontend:** Built with React 19 and Tailwind CSS, offering a highly responsive and dynamic user interface.
- **Backend:** Powered by Python's FastAPI, which efficiently handles concurrent requests, background tasks (like SMTP emails), and real-time Socket.io connections.
- **Security Layer:** Features in-transit AES-256-GCM encryption for sensitive data, ensuring that client-server communications remain secure.

CHAPTER 6

SOFTWARE ENGINEERING PARADIGM APPLIED

Agile Software Development Methodology

The project followed an Agile methodology, allowing the team to iteratively develop and deploy features. This paradigm was crucial for integrating complex features like AI embeddings and real-time sockets in stages. Continuous integration and testing enabled rapid prototyping of the AI Content Studio and the dynamic registration workflows, ensuring that stakeholder feedback could be incorporated continuously throughout the development lifecycle.

CHAPTER 7

SOFTWARE AND HARDWARE REQUIREMENT SPECIFICATIONS

SOFTWARE REQUIREMENTS

- **Frontend:** React 19, TypeScript, Vite, Tailwind CSS, Framer Motion.
- **Backend:** Python 3.11, FastAPI, Uvicorn, Python-SocketIO.
- **Database:** MongoDB Atlas (via PyMongo).
- **Third-Party Services:** Firebase (Auth), Twilio (SMS/Verify), Razorpay (Payments), Google Gemini (AI).
- **Environment:** Node.js (v18/v20) and a modern Web Browser (Chrome, Firefox, Edge).

HARDWARE REQUIREMENTS

- **Development:** Standard PC or Mac with at least 8GB RAM and an internet connection.
- **End-User (Organizers/Attendees):** Any internet-enabled device (Smartphone, Tablet, or PC). Mobile devices with functioning cameras are required for the on-site QR code ticket scanning feature.

CHAPTER 8

SYSTEM DESIGN

The system architecture of Eventron is modular and layered:

1. **Client Layer:** React SPA that interacts with the backend via HTTP REST protocols and WebSockets for real-time updates.
2. **Application Layer (FastAPI):** Routes requests, applies RBAC middleware, decrypts AES-256 payloads, and orchestrates business logic (e.g., calling Gemini API for content generation or Razorpay for payment validation).
3. **Data Access Layer:** Interacts with MongoDB to perform CRUD operations, utilizing automated indexing for fast query performance. Persistent disk caching (DiskCache) is employed to proxy event poster images and cache heavy queries, minimizing latency.
4. **Notification Engine:** Manages asynchronous multi-channel alerts (SMTP Email, Twilio SMS, VAPID Web Push).

CHAPTER 9

DATABASE DESIGN

Eventron uses a NoSQL database (MongoDB) designed for flexibility and scalability. Key collections include:

- **Users:** Stores multi-modal authentication identifiers, RBAC roles, and profile metadata.
- **Events:** Stores event details, AI-generated marketing copy, vector embeddings for similarity searches, pricing, and collaborative organizer IDs.
- **Registrations/Tickets:** Links Users to Events, storing transaction IDs (Razorpay), registration models (Individual/Team), and unique QR code metadata.
- **Teams:** Manages team rosters, unique invite codes, and associated event IDs for team-based registrations.

CHAPTER 10

RESULTS AND DISCUSSIONS

Eventron successfully delivers a fully functional, high-performance platform. Key achievements include:

- **AI Integration Success:** The AI Content Studio drastically reduces event setup time, and the vector-based recommendation engine (using Gemini text-embedding-004) significantly improves event discovery relevance for users.
- **Real-Time Efficiency:** WebSockets ensure that organizers see ticket sales and check-ins in real-time without page refreshes.
- **Security & Reliability:** The implementation of AES-256-GCM encryption and sliding-window rate limiting ensures the platform remains secure against common web vulnerabilities.

CHAPTER 11

CONCLUSION

Eventron demonstrates the immense potential of integrating AI, robust security protocols, and real-time web technologies into a single unified platform. By addressing the pain points of both event organizers and attendees, the project successfully delivers a modern alternative to traditional event management systems. The modular architecture ensures that the platform is scalable, maintainable, and ready for production-level traffic.

CHAPTER 12

FUTURE WORKS

- **Native Mobile Applications:** Development of native iOS and Android apps using React Native for deeper OS integration.
- **Augmented Reality (AR) Venue Mapping:** Allowing attendees to use AR to navigate large event venues and find specific booths or stages.
- **Advanced Analytics Dashboard:** Incorporating predictive ML models to forecast ticket sales and suggest optimal pricing strategies to organizers.
- **Blockchain Ticketing:** Exploring NFT-based tickets to completely eliminate ticket scalping and fraud.

APPENDICES

A. LIST OF TECHNICAL TERMS

- **Vector-Based Recommendations:** Using mathematical vectors (embeddings) to measure the cosine similarity between a user's past events and new events to suggest highly relevant matches.
- **Holographic Tickets:** A visually dynamic, digital ticket format featuring a scannable QR code for secure entry.
- **Web Push (VAPID):** Voluntary Application Server Identification; a secure protocol for sending push notifications directly to a user's web browser.
- **Disk Caching:** Storing frequently accessed data (like images or database queries) on the server's disk to deliver it quickly without recalculating.

B. ABBREVIATIONS USED

- **SPA:** Single Page Application
- **RBAC:** Role-Based Access Control
- **OTP:** One-Time Password
- **AES:** Advanced Encryption Standard (AES-256-GCM)
- **API:** Application Programming Interface
- **ASGI:** Asynchronous Server Gateway Interface

BIBLIOGRAPHY

- [1] React Documentation. (n.d.). React: The library for web and native user interfaces. Retrieved from <https://react.dev/>
- [2] FastAPI Documentation. (n.d.). FastAPI: Modern, fast (high-performance), web framework for building APIs with Python. Retrieved from <https://fastapi.tiangolo.com/>
- [3] MongoDB Manual. (n.d.). MongoDB Atlas Documentation. Retrieved from <https://www.mongodb.com/docs/atlas/>
- [4] Google Gemini API. (n.d.). Google AI for Developers. Retrieved from <https://ai.google.dev/>
- [5] Razorpay Docs. (n.d.). Razorpay Payment Gateway Integration. Retrieved from <https://razorpay.com/docs/>
- [6] Firebase Documentation. (n.d.). Firebase Authentication. Retrieved from <https://firebase.google.com/docs/auth>